GCA Session 4 - Lab Report

Stijn Vandendriessche
Master of Electronics and ICT Engineering Technology

Abstract: This report focusses on optimizing execution time through the strategic use of different memory types. Specifically, I studied the performance impact when using only global memory, combining with shared memory and combining with constant memory. The hypothesis was that shared and constant memory would result in a significant performance increase, since this reduces reads and writes to the slower global memory. This hypothesis proved to be true within the context of giant matrix multiplication.

Keywords: CUDA, GPU, CPU, C++

1 Concepts

This report discusses three distinct memory types: global memory, shared memory and constant memory.

1.1 Global Memory

This is the largest memory pool on the GPU. It is accessible to every thread within the entire grid, and is usually used for loading input data and temporarily storing output data. However, accessing global memory tends to be slower compared to shared and constant memory. This means that for functions with intensive read and write operations it can be beneficial to use these alternative memory types to enhance performance.

1.2 Shared Memory

This is fast, on-chip memory space that is shared among threads within a single thread block. Is is limited in size, but is significantly faster than global memory.

1.3 Constant Memory

This is a specialized read-only memory space that is optimized for storing data that remains constant throughout the execution of a kernel. It is even faster than shared memory, but is also limited in size.

2 Project

In order to illustrate the concepts discussed above, I implemented three kernels with a similar task: multiplying two very large matrices.

2.1 Global Memory Kernel

This multiplication kernel uses only shared memory, and has a very straight forward approach. It iterates over the relevant row and column, performing multiplication of the targeted numbers and adding the result to the output.

2.2 Shared Memory Kernel

This multiplication kernel uses shared memory. Due to it's limited size, a tiling mechanism is introduced. With this technique, a matrix is divided into smaller submatrices. A tile of size NxN in the output is obtained by summing the results of the corresponding matrix multiplications of NxN tiles in the inputs.

2.3 Constant Memory Kernel

The third kernel is similar to the second one, as it also uses tiling. It differs in the type of memory it uses, being constant memory.

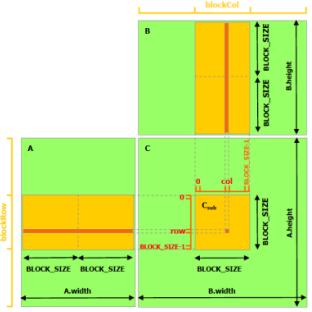


Figure 1: *Illustration of the tiling mechanism*

3 Results

The following graph illustrates the time difference between the different kernels. All kernels were executed 1000 times, and were tasked with multiplying two matrices of size 1024 by 1024. The results are presented in milliseconds. The figure shows constant memory, global memory, and shared memory from left to right.

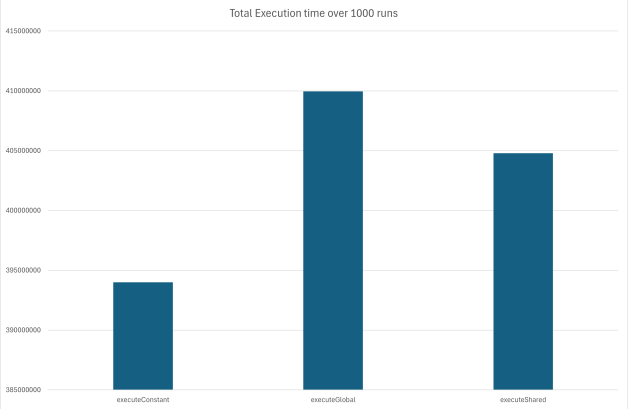


Figure 2: *Enter Caption*

4 Github link

`https://github.com/StijnVandendriessche1/GCA_lab4.git`